



南開大學
Nankai University

计算机学院
人工智能导论实验报告

实验五：口罩佩戴检测

姓名：林盛森

学号：2312631

专业：计算机科学与技术

2025 年 5 月 21 日

目录

1 问题重述	2
2 设计思想	2
3 代码内容	3
4 实验结果	6
4.1 样例测试	6
5 总结	6

1 问题重述

1.1 实验背景

今年一场席卷全球的新型冠状病毒给人们带来了沉重的生命财产的损失。有效防御这种传染病毒的方法就是积极佩戴口罩。我国对此也采取了严肃的措施，在公共场合要求人们必须佩戴口罩。在本次实验中，我们要建立一个目标检测的模型，可以识别图中的人是否佩戴了口罩。

1.2 实验要求

- 1) 建立深度学习模型，检测出图中的人是否佩戴了口罩，并将其尽可能调整到最佳状态。
- 2) 学习经典的模型 MTCNN 和 MobileNet 的结构。
- 3) 学习训练时的方法。

1.3 实验环境

可以使用基于 Python 的 OpenCV、PIL 库进行图像相关处理，使用 Numpy 库进行相关数值运算，使用 Pytorch 等深度学习框架训练模型等。

1.4 注意事项

Python 与 Python Package 的使用方式，可在右侧 API 文档中查阅。当右上角的『Python 3』长时间指示为运行中的时候，造成代码无法执行时，可以重新启动 Kernel 解决（左上角『Kernel』-『Restart Kernel』）。

1.5 参考资料

- + 论文 Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks: https://kpzhang93.github.io/MTCNN_face_detection_alignment/
- + OpenCV: https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_tutorials.html
- + PIL: <https://pillow.readthedocs.io/en/stable/>
- + Numpy: <https://www.numpy.org/>
- + Scikit-learn: <https://scikit-learn.org/>
- + PyTorch: <https://pytorch.org/>

2 设计思想

本实验旨在建立一个能够判断图中人物是否佩戴口罩的目标检测模型。整个算法设计主要分为以下几部分：数据预处理、模型构建、训练流程控制、性能优化与模型保存策略。以下是详细内容：

在数据预处理阶段，我们需要对输入的图像进行标准化统一尺寸，还需要进行图片翻转增强数据以提高模型的泛化能力，还需要进行归一化操作使模型快速收敛，划分训练集和验证集。

在模型设计方面，本实验使用轻量化网络 MobileNetV1，大幅减少了模型参数量和计算量，由于任务是二分类，模型输出层为 2 类。

在训练阶段，使用 Adam 优化器（学习率为 $1e-3$ ），具有自适应学习率调整能力，加快收敛速度。使用 CrossEntropyLoss 交叉熵损失，采用 ReduceLROnPlateau 策略，根据验证集性能动态调整学习率。

我们的任务就是需要调整一些关键参数，使得 loss 降低，最后得分提高，使得模型得到更好的训

练。

最开始直接用原代码跑的时候，测试分数只有 50 几分，我发现 epoch 设置的太低了，所以我们将其进行调整。将其增大后，模型明显训练得更好了，有 82.5 分，但还是不高。基于打印出的信息，我发现 loss 总是逐渐降低又逐渐升高，说明可能是学习率下降太快导致的，所以可以减小 factor 的值并增大 patience 的值，使得学习率下降的慢一些，这个问题可能是由于样本太少导致的。

于是我们得到修改后的参数：

```
1 train_data_loader, valid_data_loader = processing_data(data_path=data_path,
2     height=160, width=160, batch_size=128)
3 ...
4 epochs = 80
5 ...
6 scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
7     'max',
8     factor=0.3,
9     patience=5)
```

发现在这个参数模式下，我们的得分会比较高且相对稳定。

3 代码内容

```
1 # 1.加载数据并进行数据处理
2 import warnings
3 # 忽视警告
4 warnings.filterwarnings('ignore')
5
6 import cv2
7 from PIL import Image
8 import numpy as np
9 import copy
10 import matplotlib.pyplot as plt
11 from tqdm.auto import tqdm
12 import torch
13 import torch.nn as nn
14 import torch.optim as optim
15 from torchvision.datasets import ImageFolder
16 import torchvision.transforms as T
17 from torch.utils.data import DataLoader
18
19 from torch_py.Utills import plot_image
20 from torch_py.MICNN.detector import FaceDetector
21 from torch_py.MobileNetV1 import MobileNetV1
22 from torch_py.FaceRec import Recognition
23
24 def processing_data(data_path, height=224, width=224, batch_size=32,
25     test_split=0.1):
26     """
```

```

27     数据处理部分
28     :param data_path: 数据路径
29     :param height: 高度
30     :param width: 宽度
31     :param batch_size: 每次读取图片的数量
32     :param test_split: 测试集划分比例
33     :return:
34     """
35     transforms = T.Compose([
36         T.Resize((height, width)),
37         T.RandomHorizontalFlip(0.1), # 进行随机水平翻转
38         T.RandomVerticalFlip(0.1), # 进行随机垂直翻转
39         T.ToTensor(), # 转化为张量
40         T.Normalize([0], [1]), # 归一化
41     ])
42
43     dataset = ImageFolder(data_path, transform=transforms)
44     # 划分数据集
45     train_size = int((1-test_split)*len(dataset))
46     test_size = len(dataset) - train_size
47     train_dataset, test_dataset = torch.utils.data.random_split(dataset, [train_size,
48         test_size])
49     # 创建一个 DataLoader 对象
50     train_data_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
51     valid_data_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=True)
52
53     return train_data_loader, valid_data_loader
54
55 data_path = './datasets/5f680a696ec9b83bb0037081-momodel/data/image'
56
57 # 2.如果有预训练模型，则加载预训练模型；如果没有则不需要加载
58 pnet_path = "./torch_py/MTCNN/weights/pnet.npy"
59 rnet_path = "./torch_py/MTCNN/weights/rnet.npy"
60 onet_path = "./torch_py/MTCNN/weights/onet.npy"
61
62 # 3.创建模型和训练模型，训练模型时尽量将模型保存在 results 文件夹
63
64 # 加载 MobileNet 的预训练模型权
65 device = torch.device("cuda:0") if torch.cuda.is_available() else torch.device("cpu")
66 train_data_loader, valid_data_loader = processing_data(data_path=data_path,
67     height=160, width=160, batch_size=128)
68
69 epochs = 80
70 model = MobileNetV1(classes=2).to(device)
71 optimizer = optim.Adam(model.parameters(), lr=1e-3) # 优化器
72 print('加载完成...')
73 # 学习率下降的方式，acc三次不下降就下降学习率继续训练，衰减学习率

```

```

74 scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
75                                                     'max',
76                                                     factor=0.3,
77                                                     patience=5)
78 # 损失函数
79 criterion = nn.CrossEntropyLoss()
80
81 best_loss = 1e9
82 best_model_weights = copy.deepcopy(model.state_dict())
83 loss_list = [] # 存储损失函数值
84 for epoch in range(epochs):
85     model.train()
86
87     for batch_idx, (x, y) in tqdm(enumerate(train_data_loader, 1)):
88         x = x.to(device)
89         y = y.to(device)
90         pred_y = model(x)
91
92         # print(pred_y.shape)
93         # print(y.shape)
94
95         loss = criterion(pred_y, y)
96         optimizer.zero_grad()
97         loss.backward()
98         optimizer.step()
99
100        if loss < best_loss:
101            best_model_weights = copy.deepcopy(model.state_dict())
102            best_loss = loss
103
104        loss_list.append(loss)
105
106        print('step:' + str(epoch + 1) + '/' + str(epochs) + ' || Total Loss: %.4f' %
              (loss))
107 torch.save(model.state_dict(), './results/temp7.pth')
108 print('Finish Training.')
109
110 model_path = 'results/temp6.pth'
111
112 def predict(img):
113     """
114     加载模型和模型预测
115     :param img: cv2.imread 图像
116     :return: 预测的图片中的总人数、其中佩戴口罩的人数
117     """
118     # ----- 实现模型预测部分的代码 -----
119     # 将 cv2.imread 图像转化为 PIL.Image 图像，用来兼容测试输入的 cv2
120     # 读取的图像（勿删!!!）
121     # cv2.imread 读取图像的类型是 numpy.ndarray

```

```
121 # PIL.Image.open 读取图像的类型是 PIL.JpegImagePlugin.JpegImageFile
122 if isinstance(img, np.ndarray):
123     # 转化为 PIL.JpegImagePlugin.JpegImageFile 类型
124     img = Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
125
126 recognize = Recognition(model_path)
127 img, all_num, mask_num = recognize.mask_recognize(img)
128 # -----
129 return all_num, mask_num
```

4 实验结果

4.1 样例测试

📁 jupyter

接口测试

✅ 接口测试通过。

用例测试

测试点	状态	时长	结果
在 5 张图片上测试模型	✅	4s	得分:92.5

提交结果

图 4.1: 样例测试结果

可以看到我们通过了样例测试，最终得分拿到了 92.5 分。最好的模型保存在 `model_path = 'results/temp6.pth'` 中。

5 总结

在本次实验中，我成功完成了一个基于深度学习的口罩佩戴识别模型的训练与测试工作。通过对数据进行预处理、模型构建与优化，我对目标检测与图像分类任务的整体流程有了更深入的理解。

在这个过程中，调整参数的时间远远超过了读懂代码的时间，感觉这个过程确实很难。其中很大一部分原因可能是因为数据集太小了导致前后多次结果总是差距很大。

总之，通过本次实验，我成功训练出了能够区分“戴口罩”和“未戴口罩”图像的分类模型，准确率达到了预期水平。